Desarrollo de Software para Móviles DSM941 G01T

Foro 1: Jetpack Compose

Integrantes del equipo:

Nombres	Numero de Carnet
Miguel Eduardo Vallejos Linares	VL131638
Alejandro Benjamín Rivera Ochoa	RO243154
Mayron Steve Lopez Girón	LG243153
Andrea Paola Montenegro Paz	MP101106
Marlon Eduardo Montenegro Paz	MP243207

Docente: Ing. Alexander Sigüenza

Fecha de entrega: 26/04/2026

Link Video: DSM.mp4

Link GitHub: https://github.com/MarlonMontenegro/GradesApp

Contenido

Introducción 3
¿Qué es Jetpack Compose? 4
Ventajas de Jetpack Compose 5
Descripción del flujo implementado y decisiones de diseño 6
Conclusiones 7
Referencias 8

Introducción

El desarrollo de aplicaciones móviles ha evolucionado significativamente en los últimos años, impulsado por la necesidad de crear interfaces más dinámicas, eficientes y fáciles de mantener. En el ecosistema Android, el lenguaje Kotlin se ha consolidado como una herramienta clave para el desarrollo moderno, acompañado de tecnologías que buscan optimizar la construcción de interfaces de usuario.

Dentro de este contexto surge Jetpack Compose, una solución propuesta por Google que introduce un enfoque declarativo para el diseño de interfaces. A diferencia del modelo tradicional basado en XML, Compose permite construir componentes visuales directamente desde código Kotlin, lo que contribuye a una mayor claridad y reducción de complejidad en los proyectos.

El presente documento tiene como propósito analizar los fundamentos de Jetpack Compose, identificar sus principales ventajas y describir su aplicación en el desarrollo de una solución móvil sencilla. Dicha solución permite gestionar el inicio de sesión de un usuario, ingresar notas académicas y calcular automáticamente el promedio, determinando si el estudiante ha aprobado o reprobado.

¿Qué es Jetpack Compose?

Jetpack Compose es un toolkit moderno para la creación de interfaces de usuario en aplicaciones Android. Su principal característica es la adopción de un paradigma declarativo, en el cual la interfaz se define como una función del estado de la aplicación. Esto implica que la representación visual cambia automáticamente cuando los datos se modifican.

En este modelo, los elementos de la interfaz se construyen mediante funciones anotadas con `@Composable`, las cuales pueden combinarse para formar estructuras más complejas. Este enfoque permite desarrollar interfaces más organizadas, reutilizables y fáciles de mantener.

Además, Jetpack Compose se integra completamente con Kotlin, lo que permite aprovechar sus características modernas y facilita la implementación de aplicaciones más eficientes.

Ventajas de Jetpack Compose

El uso de Jetpack Compose presenta múltiples beneficios en comparación con el enfoque tradicional basado en XML. En primer lugar, reduce la cantidad de código necesario para diseñar interfaces, ya que elimina la separación entre lógica y diseño en archivos distintos.

Asimismo, mejora la mantenibilidad del código, debido a que el modelo declarativo permite que la interfaz se actualice automáticamente en función de los cambios en el estado de la aplicación. Esto reduce la probabilidad de errores y simplifica la gestión de la interfaz.

Otro aspecto relevante es la reutilización de componentes. Las funciones composables pueden emplearse en distintas partes del proyecto, lo que favorece la organización del código y evita duplicaciones innecesarias.

Finalmente, Jetpack Compose facilita la creación de interfaces dinámicas, ya que está diseñado para reaccionar de forma inmediata a los cambios en los datos, mejorando la experiencia del usuario.

Descripción del flujo implementado y decisiones de diseño

La aplicación desarrollada sigue un flujo estructurado en cuatro etapas principales. En primer lugar, el usuario accede a una pantalla de autenticación donde debe ingresar su correo electrónico y contraseña. En esta etapa se aplican validaciones básicas para asegurar que los datos sean correctos.

Posteriormente, se muestra una pantalla de bienvenida en la que se presenta información del usuario, funcionando como una transición hacia el módulo principal de la aplicación.

En la tercera etapa, el usuario ingresa las notas académicas. El sistema permite introducir al menos tres valores numéricos y realiza las validaciones necesarias para garantizar que los datos sean válidos.

Finalmente, la aplicación calcula el promedio de las notas ingresadas y muestra un resultado en pantalla, indicando si el estudiante ha aprobado o reprobado según un criterio previamente definido.

En cuanto a las decisiones de diseño, se optó por utilizar Kotlin como lenguaje principal debido a su integración con Jetpack Compose. Asimismo, se eligió este toolkit por su enfoque declarativo, que simplifica la construcción de interfaces.

También se decidió implementar validaciones en cada etapa del flujo para evitar errores en la ejecución. Además, se estructuró la aplicación en múltiples pantallas para mejorar la organización y facilitar el mantenimiento del código.

Conclusiones

Jetpack Compose representa una evolución importante en el desarrollo de interfaces para aplicaciones Android, al introducir un enfoque declarativo que simplifica la construcción y actualización de la interfaz.

Durante el desarrollo de la aplicación, se pudo observar que esta herramienta permite trabajar de forma más organizada, reduciendo la cantidad de código y mejorando la claridad del proyecto.

Además, su integración con Kotlin facilita el uso de características modernas del lenguaje, lo que contribuye a un desarrollo más eficiente y estructurado.

En conclusión, Jetpack Compose se posiciona como una alternativa sólida para el desarrollo de aplicaciones móviles, especialmente en proyectos que requieren simplicidad, mantenimiento eficiente y rapidez en la implementación.

Referencias

Google. (2024). *Jetpack Compose*. https://developer.android.com/compose

Kotlin. (2024). *Kotlin Programming Language*. https://kotlinlang.org